

如何在 Gemini 使用函式呼叫與 API 互動

來源：<https://codelabs.developers.google.com/codelabs/gemini-function-calling?hl=zh-tw>

步驟數：12

在這個程式碼研究室中，您將使用 Gemini 函式呼叫建構應用程式，讓使用者詢問匯率、從外部 API 擷取最新資料，然後找出答案來回覆使用者。

目錄

1. [總覽](#)
2. [設定和需求](#)
3. [瞭解問題](#)
4. [嘗試常見的解決方法](#)
5. [函式呼叫的運作方式](#)
6. [選擇 API](#)
7. [定義函式和工具](#)
8. [生成函式呼叫](#)
9. [提出 API 要求](#)
10. [生成回覆](#)
11. [查看完整程式碼範例](#)
12. [恭喜](#)

步驟 1 · 約 2 分鐘

1. 總覽

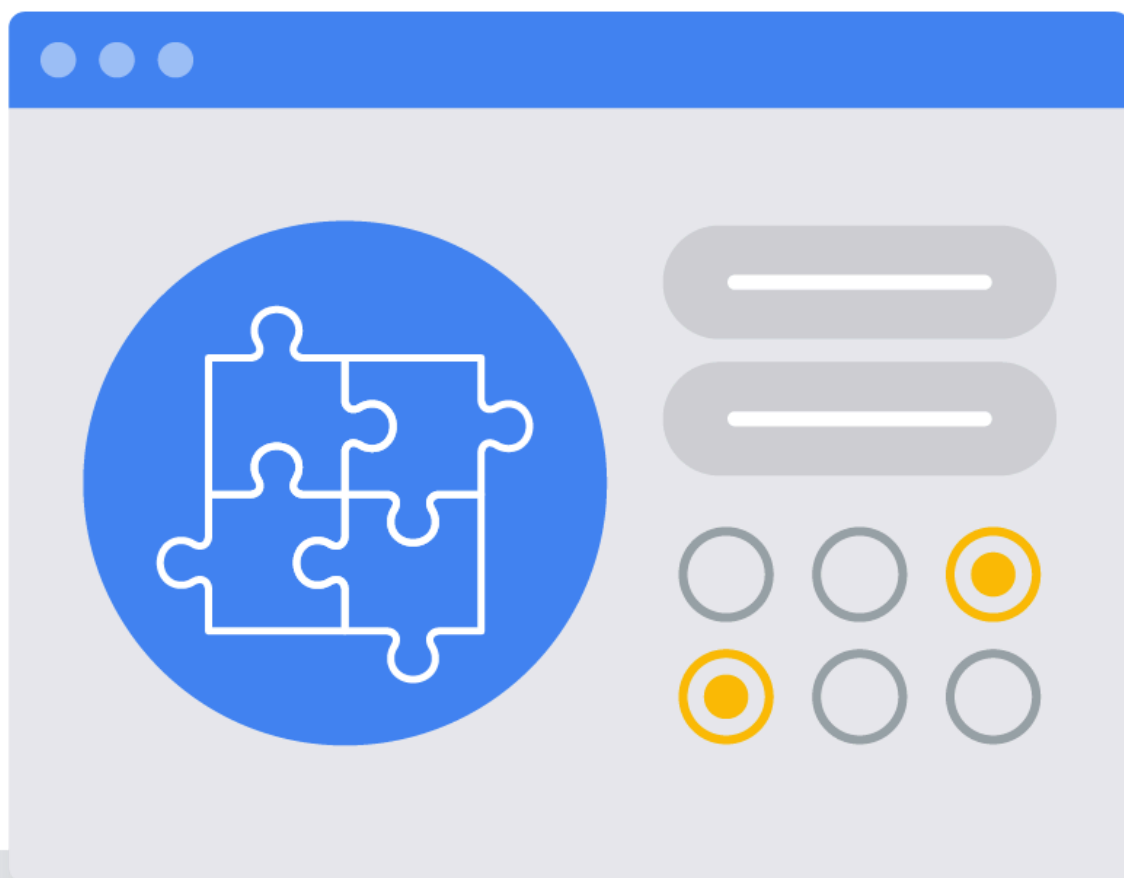
Gemini 的函式呼叫是什麼？

[Vertex AI Gemini API](#) 是由 Google DeepMind 開發的一系列生成式 AI 模型，專為多模態用途而設計。[函式呼叫](#)是 Gemini 模型的功能，可協助開發人員更輕鬆地從生成模型取得結構化資料輸出內容。

開發人員接著可以使用這些輸出內容呼叫其他 API，並將相關回應資料傳回模型。換句話說，函式呼叫可協助您將生成模型連結至外部系統，確保生成的內容包含最新且準確的資訊。

函式呼叫的運作方式

函式是使用函式宣告來描述，這有助於生成模型瞭解函式中的用途和參數。在查詢中將函式宣告傳遞至生成模型後，模型會根據使用者的查詢傳回結構化物件，其中包含相關函式的名稱和引數。請注意，使用函式呼叫時，模型不會實際呼叫函式，您可以改用傳回的函式和參數，在任何語言、程式庫或架構中呼叫函式！



建構項目

在本程式碼研究室中，您將使用 Vertex AI Gemini API 和 Python 建構生成式 AI pipeline。使用者可以透過您的應用程式查詢匯率，系統會從外部 API 擷取最新資料，並回覆使用者。

課程內容

- 如何使用 Python 用戶端程式庫與 Gemini 模型互動
- 如何定義函式宣告並註冊為工具

- 如何呼叫 Gemini 並取得函式呼叫回應
- 如何將函式回應傳回 Gemini 並回覆使用者

軟硬體需求

- Google Cloud 專案
 - 瀏覽器，例如 [Chrome](#)
 - Python 開發環境，例如 [Colab](#) 或 [Colab Enterprise](#)
-

步驟 2 · 約 2 分鐘

2. 設定和需求

如要開始在 Gemini 中使用函式呼叫，請先啟用 Vertex AI API，並安裝最新版 Vertex AI Python 用戶端程式庫。

啟用 Vertex AI API

如要啟用 Vertex AI API，請按照下列步驟操作：

1. 在瀏覽器中前往 [Vertex AI API 服務詳細資料頁面](#)。
2. 點選「啟用」按鈕，在 Google Cloud 專案中啟用 Vertex AI API。

安裝 Vertex AI 的 Python 用戶端程式庫

如要安裝 Vertex AI 的 Python 用戶端程式庫，請按照下列步驟操作：

1. 在開發環境中開啟終端機。
2. 確認您有有效的 Python 開發環境，並視需要參閱這些[指南](#)。
3. 執行下列指令，安裝 Vertex AI 的 Python 用戶端程式庫：

```
pip install --upgrade google-cloud-aiplatform
```

4. 如果您在筆記本環境中執行，可能需要重新啟動執行階段/核心，才能使用新安裝的套件。

現在可以使用 Vertex AI API 了！

3. 瞭解問題

您是否曾與大型語言模型或生成式 AI 模型互動，詢問即時或當前資訊，但模型卻回覆過時或不準確的資訊？

現在就來試試看！首先，我們會匯入相關的 Python 套件，並初始化 Gemini 模型。您可以在 [Colab](#) 或 [Colab Enterprise](#) 等 Python 開發環境中執行下列程式碼，並安裝最新版本的 [Vertex AI SDK for Python](#)：

```
import vertexai
from vertexai.generative_models import GenerativeModel
model = GenerativeModel("gemini-1.5-pro-001")
```

現在，讓我們詢問今天不同貨幣的匯率：

```
response = model.generate_content(
    "What's the exchange rate for euros to dollars today?"
)
print(response.text)
```

模型應會生成有限或過時的回覆，類似於：

```
As an AI language model, I don't have access to real-time currency exchange
rates. However, as of my last update in September 2021, the approximate exchange
rate between euros (EUR) and US dollars (USD) was:
```

```
1 EUR ≈ 1.18 USD
```

```
Please note that currency exchange rates constantly fluctuate and can vary
depending on various factors such as economic conditions, supply and demand,
political events, and more. To obtain the most up-to-date and accurate exchange
rate, I recommend using a reliable currency converter or financial website that
provides live rates.
```

```
[...]
```

如果使用者收到這類回覆，就必須切換情境來查詢感興趣的貨幣、取得最新匯率，並自行執行任何轉換。

理想情況下，生成模型管道可以為使用者處理部分或所有這些工作。在下一節中，您將嘗試一些常見的解決方法，從生成模型取得結構化回覆，以便呼叫外部系統。

4. 嘗試常見的解決方法

在需要最新資訊或外部來源資料的生成模型使用情境中，您可以呼叫外部 API，然後將結果傳回生成模型，供模型用於回覆。

呼叫外部系統前，您需要判斷要使用的正確函式、從使用者擷取相關參數，並將參數放入結構化資料物件。這通常需要詳盡的提示工程，強迫生成模型輸出有效的結構化資料。

讓我們回到上一節提出的問題，並為模型新增一些額外指示。請嘗試將下列要求傳送至 Gemini 模型：

```
user_prompt = "What's the exchange rate from euros to US dollars today?"

response = model.generate_content("""
Your task is to extract parameters from the user's input and return it as a
structured JSON payload. The user will ask about the exchange rate and which
currency they are converting from and converting to.

User input: {user_prompt}

Please extract the currencies as parameters and put them in a JSON object.
""").format(user_prompt=user_prompt)
print(response.text)
```

這會產生下列文字回覆，但這不是有效的 JSON，我們很難處理：

```
```json
{
 "currency_from": "euros",
 "currency_to": "US dollars"
}
```
```

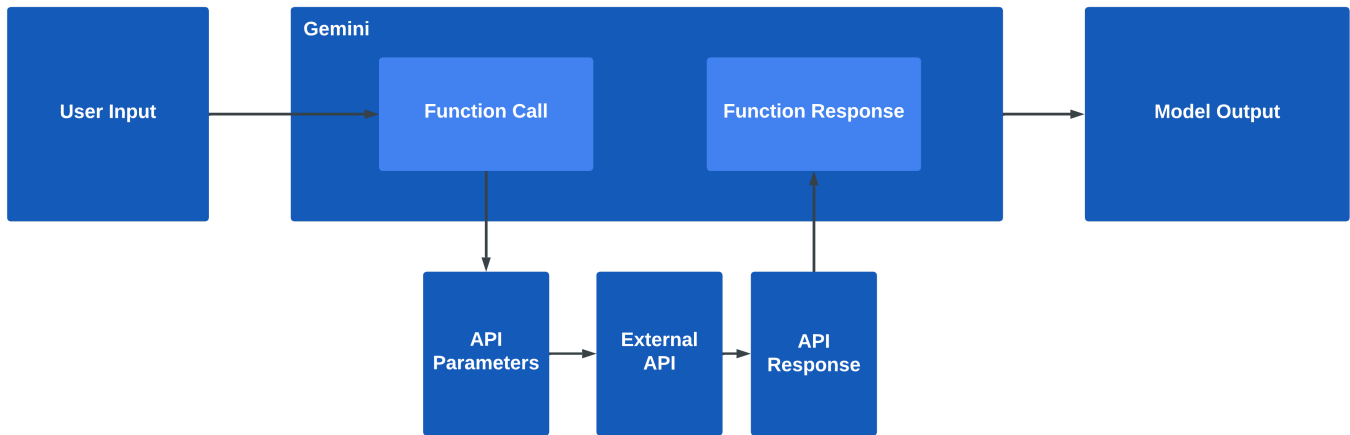
具體來說，文字回覆的第一行和最後一行包含反引號，用於分隔程式碼區塊；第一行包含語言指定符；JSON 物件中的值並非標準的三字母貨幣縮寫，而貨幣匯率 API 會將這類縮寫視為輸入參數。

我們可以嘗試使用 Python 將這段文字後續處理為有效的 JSON 和字典、在提示中新增更多指令、提供一或多個所需輸出內容的範例、微調模型，或是再次呼叫生成模型，要求清理 JSON。

但其實有更確定的方法！讓我們瞭解如何在 Gemini 中使用函式呼叫，查詢外部服務中的資訊，並向使用者傳回相關回覆。

5. 函式呼叫的運作方式

開始擷取參數和呼叫函式前，請先瞭解函式呼叫的步驟，以及在執行階段使用的元件。



使用者輸入內容至 Gemini API

系統會將使用者的提示傳送至 Gemini API，而在對 Gemini 模型發出的 API 呼叫中，開發人員已在工具中定義一或多個函式宣告，讓 Gemini 模型知道可以呼叫哪些函式，以及如何呼叫這些函式。

使用者輸入 -> Gemini API -> 函式呼叫

Gemini API 會傳回函式呼叫

根據使用者輸入內容和提示，Gemini 會傳回函式呼叫回應，其中包含結構化資料，包括要呼叫的函式名稱和要使用的對應參數。

Gemini API -> 函式呼叫

提出 API 要求

接著，您會使用函式名稱和參數發出 API 要求，從外部系統或 API 擷取資訊。這項 API 要求和回應是由開發人員在應用程式碼中實作，且不屬於 Gemini API 和 SDK 的範圍。舉例來說，您可能會在 Python 中使用 `requests` 程式庫呼叫 REST API，並接收 JSON 回應。或者，您可以使用偏好的方法和用戶端程式庫呼叫函式。

API 參數 -> 外部 API -> API 回應

將 API 回應傳回 Gemini

最後，您會將 API 回應傳回 Gemini 模型，以便生成對應於使用者初始提示的回覆，或在 Gemini 模型判斷需要額外資訊時，叫用其他函式呼叫回應。

步驟 6 · 約 0 分鐘

6. 選擇 API

您已瞭解函式呼叫的整體流程和具體步驟，現在要建構生成式 AI pipeline，擷取最新的匯率。首先，我們需要選取要用來做為資訊來源的 API。

以匯率換算應用程式為例，我們會使用 <https://www.frankfurter.app/> 的 REST API，擷取全球匯率的最新資訊。

如要與這個 REST API 互動，我們可能會在 Python 中使用 `requests` 發出 REST API 呼叫，如下所示：

```
import requests
url = "https://api.frankfurter.app/latest"
response = requests.get(url)
response.text
```

或 `CURL` 要求，例如：

```
curl https://api.frankfurter.app/latest
```

這會傳回類似以下的回應：

```
{
  "amount": 1,
  "base": "EUR",
  "date": "2023-12-20",
  "rates": {
    "AUD": 1.6186, "BGN": 1.9558, "BRL": 5.3287,
    "CAD": 1.4609, "CHF": 0.946, "CNY": 7.8121,
    "CZK": 24.538, "DKK": 7.4565, "GBP": 0.86555,
    "HKD": 8.5439, "HUF": 385.23, "IDR": 16994,
    "ILS": 3.9983, "INR": 91.06, "ISK": 150.3,
    "JPY": 157.12, "KRW": 1425.62, "MXN": 18.6867,
    "MYR": 5.0977, "NOK": 11.2895, "NZD": 1.7421,
    "PHP": 60.991, "PLN": 4.3413, "RON": 4.9699,
    "SEK": 11.129, "SGD": 1.4562, "THB": 38.252,
    "TRY": 31.883, "USD": 1.0944, "ZAR": 20.111
  }
}
```

由於 Gemini 的函式呼叫功能不會實際為您呼叫外部 API，因此您使用的 API 類型沒有這類限制！您可以使用 Cloud Run 服務、Cloud Function、對 Google Cloud 服務的 API 要求，或任何外部 REST API。

7. 定義函式和工具

選取要使用的 REST API 後，我們現在可以定義 API 規格，並在工具中註冊函式。

確認您已安裝最新版的 [Vertex AI SDK for Python](#)。

接著，從 Python SDK 匯入必要模組，並初始化 Gemini 模型：

```
from vertexai.generative_models import (
    Content,
    FunctionDeclaration,
    GenerativeModel,
    Part,
    Tool,
)

model = GenerativeModel("gemini-1.5-pro-001")
```

回到 <https://api.frankfurter.app/> 的 REST API，我們可以看到它接受下列輸入參數：

| 參數 | 類型 | 說明 |
|------|----|----------|
| from | 字串 | 要轉換的幣別 |
| to | 字串 | 要轉換成的幣別 |
| date | 字串 | 要擷取匯率的日期 |

使用這些參數時，這個 REST API 的部分 OpenAPI 規格 (YAML 格式) 如下所示：

```
openapi: 3.0.0
info:
  title: Frankfurter Exchange Rate API
  description: This API provides current and historical exchange rates
  version: 1.0.0
servers:
  - url: https://api.frankfurter.app
paths:
  /{date}:
    get:
      summary: Get the latest currency exchange rates.
      parameters:
        - name: date
          in: path
          description: Get currency rates for a specific date or 'latest' if a date is not
specified
          required: true
          schema:
            type: string
        - name: from
          in: query
          description: The currency to convert from.
          required: true
          schema:
            type: string
        - name: to
          in: query
          description: The currency to convert to.
          schema:
            type: string
```

現在，請使用 Gemini 的 Python SDK，將這項工具登錄為 `FunctionDeclaration`：

```

get_exchange_rate_func = FunctionDeclaration(
    name="get_exchange_rate",
    description="Get the exchange rate for currencies between countries",
    parameters={
        "type": "object",
        "properties": {
            "currency_date": {
                "type": "string",
                "description": "A date that must always be in YYYY-MM-DD format or the value
'latest' if a time period is not specified"
            },
            "currency_from": {
                "type": "string",
                "description": "The currency to convert from in ISO 4217 format"
            },
            "currency_to": {
                "type": "string",
                "description": "The currency to convert to in ISO 4217 format"
            }
        },
        "required": [
            "currency_from",
            "currency_date",
        ]
    },
)

```

請務必在函式和參數說明中盡可能提供詳細資訊，因為生成模型會使用這些資訊判斷要選取哪個函式，以及如何在函式呼叫中填入參數。

最後，您將定義包含函式宣告的 `Tool`：

```

exchange_rate_tool = Tool(
    function_declarations=[get_exchange_rate_func],
)

```

您可以在工具中使用一個函式宣告，但請注意，您可以在工具中註冊一或多個函式宣告，模型會在執行階段選取要使用的適當函式。如要進一步瞭解 Gemini SDK for Python 中的 `FunctionDeclaration`、`Tool` 和相關類別，請參閱 [Gemini API 中的函式呼叫](#) 說明文件。

您已完成函式和工具定義的設定。在下一節中，我們會使用這項工具呼叫生成模型，並取得可用於呼叫 REST API 的函式呼叫。

步驟 8 · 約 0 分鐘

8. 生成函式呼叫

現在您可以提示生成模型，並加入您定義的 `tool`：

```
prompt = """What is the exchange rate from Australian dollars to Swedish krona?
How much is 500 Australian dollars worth in Swedish krona?"""

response = model.generate_content(
    prompt,
    tools=[exchange_rate_tool],
)
```

讓我們來看看回應物件：

```
print(response.candidates[0].content)

role: "model"
parts {
  function_call {
    name: "get_exchange_rate"
    args {
      fields {
        key: "currency_to"
        value {
          string_value: "SEK"
        }
      }
      fields {
        key: "currency_from"
        value {
          string_value: "AUD"
        }
      }
      fields {
        key: "currency_date"
        value {
          string_value: "latest"
        }
      }
    }
  }
}
```

模型似乎選取了可用的函式，並傳回 `get_exchange_rate` 函式的函式呼叫和參數。且參數格式正確。恭喜！您已成功從生成模型取得結構化回覆。

在下一節中，您將使用回應中的資訊提出 API 要求。

步驟 9 · 約 0 分鐘

9. 提出 API 要求

請注意，Gemini 中的函式呼叫功能不會實際為您呼叫外部 API。您可以自由使用任何語言、程式庫或架構！

您將在 Python 中使用 `requests` 程式庫呼叫匯率 REST API。

讓我們將回應解壓縮至 Python 字典：

```
params = {}
for key, value in response.candidates[0].content.parts[0].function_call.args.items():
    params[key[9:]] = value
params
```

現在我們可以呼叫 `requests` 或任何其他方法：

```
import requests
url = f"https://api.frankfurter.app/{params['date']}"
api_response = requests.get(url, params=params)
api_response.text
```

這會產生類似以下的回應：

```
'{"amount":1.0,"base":"AUD","date":"2024-01-16","rates":{"SEK":6.8682}}'
```

我們已收到 REST API 的回應，其中包含今天的最新匯率資訊。在下一節中，我們會將這項資訊傳回模型，以便為使用者生成相關回應。

10. 生成回覆

最後，在下一個對話回合中，將函式回應傳回模型，為使用者生成回覆：

```
response = model.generate_content([
    Content(role="user", parts=[
        Part.from_text(prompt + ""Give your answer in steps with lots of detail
            and context, including the exchange rate and date."""),
    ]),
    Content(role="function", parts=[
        Part.from_dict({
            "function_call": {
                "name": "get_exchange_rate",
            }
        })
    ]),
    Content(role="function", parts=[
        Part.from_function_response(
            name="get_exchange_rate",
            response={
                "content": api_response.text,
            }
        )
    ]),
],
tools=[exchange_rate_tool],
)

response.candidates[0].content.parts[0].text
```

將函式回應傳回模型後，模型就會回應使用者的提示，並附上 API 回應中的相關資訊。

The exchange rate from Australian dollars to Swedish krona on January 16, 2024, is 1 Australian dollar is equal to 6.8663 Swedish krona.

So, 500 Australian dollars would be worth $500 * 6.8663 = 3,433.15$ Swedish krona.

步驟 11 · 約 2 分鐘

11. 查看完整程式碼範例

此時，您可以使用 Cloud Run 服務、Cloud Function 或其他 Cloud 服務，將 Python 程式碼放在後端 API 中，並部署前端應用程式，透過這個後端 API 執行模型查詢和 API 呼叫。

以下是最終解決方案的完整程式碼範例：

```

import requests
from vertexai.generative_models import (
    Content,
    FunctionDeclaration,
    GenerativeModel,
    Part,
    Tool,
)

model = GenerativeModel("gemini-1.5-pro-001")

get_exchange_rate_func = FunctionDeclaration(
    name="get_exchange_rate",
    description="Get the exchange rate for currencies between countries",
    parameters={
        "type": "object",
        "properties": {
            "currency_date": {
                "type": "string",
                "description": "A date that must always be in YYYY-MM-DD format or the value
'latest' if a time period is not specified"
            },
            "currency_from": {
                "type": "string",
                "description": "The currency to convert from in ISO 4217 format"
            },
            "currency_to": {
                "type": "string",
                "description": "The currency to convert to in ISO 4217 format"
            }
        },
        "required": [
            "currency_from",
            "currency_date",
        ]
    },
)

exchange_rate_tool = Tool(
    function_declarations=[get_exchange_rate_func],
)

prompt = """"What is the exchange rate from Australian dollars to Swedish krona?
How much is 500 Australian dollars worth in Swedish krona?"""

response = model.generate_content(
    prompt,
    tools=[exchange_rate_tool],
)

response.candidates[0].content

```

```

params = {}
for key, value in response.candidates[0].content.parts[0].function_call.args.items():
    params[key[9:]] = value
params

import requests
url = f"https://api.frankfurter.app/{params['date']}"
api_response = requests.get(url, params=params)
api_response.text

response = model.generate_content(
    [
        Content(role="user", parts=[
            Part.from_text(prompt + "" "Give your answer in steps with lots of detail
            and context, including the exchange rate and date."""),
        ]),
        Content(role="function", parts=[
            Part.from_dict({
                "function_call": {
                    "name": "get_exchange_rate",
                }
            })
        ]),
        Content(role="function", parts=[
            Part.from_function_response(
                name="get_exchange_rate",
                response={
                    "content": api_response.text,
                }
            )
        ]),
    ],
    tools=[exchange_rate_tool],
)

response.candidates[0].content.parts[0].text

```

在這個實作中，我們對生成模型發出兩項要求：一項要求生成函式呼叫，另一項要求傳回函式回應。請注意，這只是使用 Gemini 處理函式呼叫和函式回應的方法之一。您也可以進行額外的函式呼叫，取得查詢的更多資訊，或搭配聊天和非同步方法使用函式呼叫。

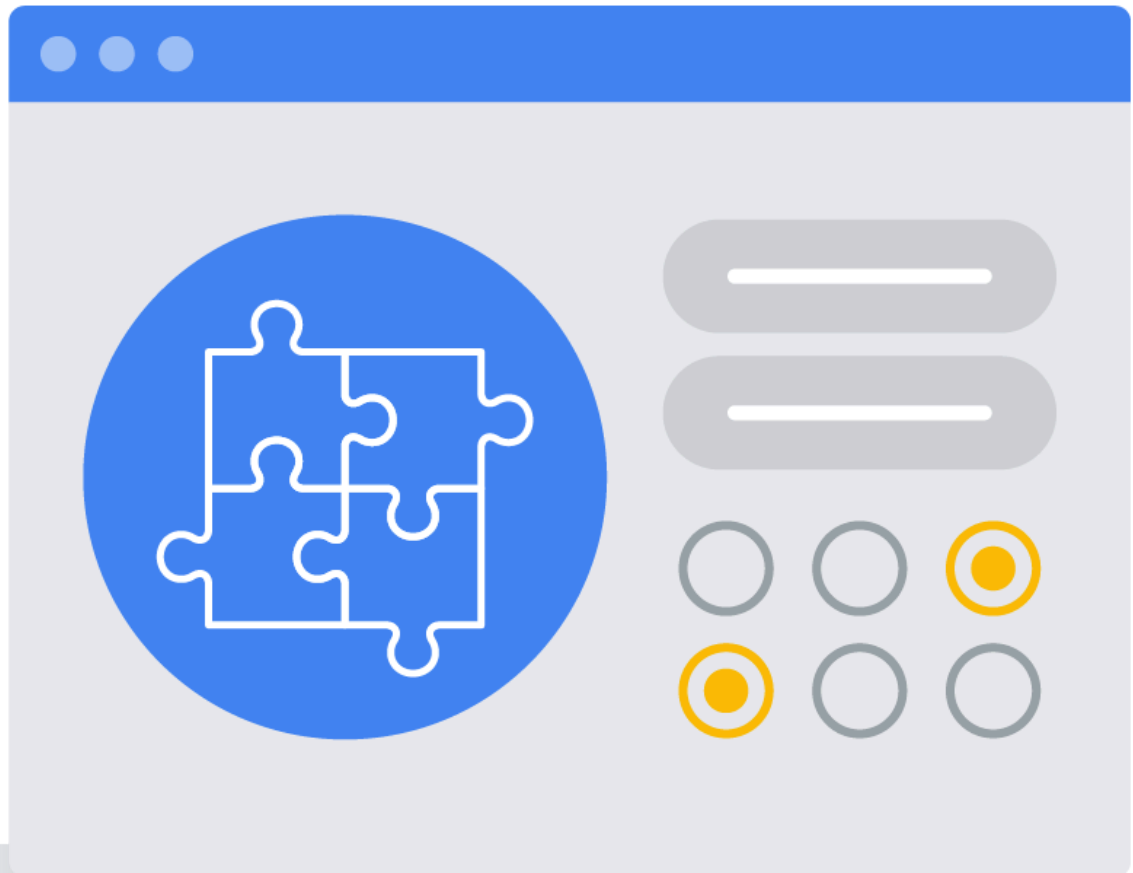
如需其他程式碼範例，請參閱 [Gemini 函式呼叫的範例筆記本](#)。

12. 恭喜

您已成功使用 Gemini 的函式呼叫功能，建構生成式 AI 管道，並搭配 Vertex AI Gemini API 和 Python 使用！使用者可以詢問匯率，系統會從外部 API 擷取最新資料，並提供回覆。

收到使用者的提示後，Gemini 函式呼叫會負責選取適當的函式、從提示中擷取參數，並傳回結構化資料物件，供您發出外部 API 呼叫。

Gemini 的函式呼叫設計旨在提供兩全其美的解決方案，讓您能以決定性方式擷取參數，同時將摘要和內容建立作業交給生成模型。歡迎在管道中試用其他 API 和提示，並探索與 [Vertex AI Gemini API](#) 相關的其他功能。



清除

如要避免系統向您的 Google Cloud 帳戶收取本程式碼研究室所用資源的費用，請執行下列清理作業：

- 請前往 [Google Cloud 控制台](#) 刪除您不需要的專案，以免產生不必要的 Google Cloud 費用。
- 如要停用 Vertex AI 的 API，請前往 [Vertex AI API 服務詳細資料頁面](#)，然後按一下「停用 API」並確認。

瞭解詳情

歡迎參考下列指南和資源，進一步瞭解對話式 AI 和生成式 AI：

- [Gemini 模型簡介](#)
- [Gemini 函式呼叫說明文件](#)
- [Gemini 函式呼叫的範例筆記本](#)

- [Gemini 範例筆記本](#)
- [Google Cloud 中的生成式 AI](#)

授權

這項內容採用的授權為 [Creative Commons 姓名標示 2.0 通用授權](#)。
